

Simulation of electromagnetic waves using Finite-Difference Time-Domain (FDTD)

Nicolas Ferreira

Today

Abstract

This thesis presents an alternative implementation and notation for the Finite-Differences Time-Domain method for the simulation of electromagnetic waves by translating the discretized derivatives as tensor operations. In contrast, other approaches may include array shifts, for-loops or more classical linear algebra. The main goal of this thesis is a modular and user-friendly Python class to be used along with Matplotlib for academic purposes. The goal of the approach is, however, to test the simplicity of its implementation and its performance. The idea - or hope - behind it is that the ever-evolving machine learning libraries may help provide a simple interface for highly optimized tensor operations. Further studies within this thesis includes the comparison between ideal Fresnel coefficients and its simulated counterparts. This was done through measurements of power of incoming, reflected and refracted waves. It was found, however, that this numerical simulation does not guarantee conservation of energy, especially when both abrupt changes in material and high frequencies are involved. As already available wave sources, point, line, gaussian-beam and phased-array options are pre-programmed.

Contents

1	Introduction	5
2	Electromagnetic theory and wave propagation	7
2.0.1	The Maxwell Equations	7
2.0.2	Ohm's Law	7
2.1	The electromagnetic wave equations	8
2.1.1	Homogeneous equations	8
2.1.2	Non-homogeneous equations	8
2.2	Propagation from sources	9
2.2.1	Necessary equations for the propagation	11
3	FDTD - Yee's method	13
3.1	Finite differences	13
3.1.1	Central differences	13
3.2	Yee cells	14
3.2.1	TE-Propagation	14
3.2.2	TM-Propagation	15
3.2.3	2D Yee Cell	16
3.3	Discretization in 2D	17
3.3.1	Faraday's Law	17
3.3.2	Ampere-Maxwell	18
3.4	Discretization	18
3.4.1	TE Waves	19
3.4.2	TM Waves	19
3.4.3	Update Equations	20
4	Implementation through tensors	21
4.1	Updated Update Equations	21
4.2	Fields and operators	22
4.2.1	Code so far	23
4.2.2	Operators for $H_{x i j}^q$	24
4.2.3	Operator for $H_{y i j}^q$	24
4.2.4	Operator for $E_{z i j}^q$	25
4.2.5	Code so far	26

4.3	The constants and materials	27
4.3.1	Propagation constants	28
4.4	Final symbolic formulation	28
4.4.1	Code so far	29
5	Mur's Absorbing Boundary Condition	33
5.1	Derivation of Mur's ABC	33
5.1.1	Code so far	35
6	Stability and units of measurement	37
6.1	Stability condition	37
6.2	Units of measurement	39
6.3	Normalising units	39
6.3.1	Time and length	39
6.4	Multilinear maps	41
6.4.1	Definition	41
6.4.2	Unique characterization and correspondence to a tensor	41

Chapter 1

Introduction

Since the development of computers, all sciences have relied on them to approximate solutions to the initial value problems and differential equations describing the laws of nature. Electromagnetic waves are no exception. In 1966, mathematician Kane Yee published his work *"Numerical solution of initial boundary value problems involving maxwell's equations in isotropic media"*, where he applied the a leapfrog time integration on Maxwell's equations discretized as finite-difference equations. This algorithm now goes by the name Finite Difference Time-Domain (FDTD), or Yee's algorithm. FDTD has proven to be extremely useful throughout the years as well as simple to implement, and it has thus been used from mere visualization of the behaviour of electromagnetic waves, to studies of new waveguide technologies.



The principles of FDTD remained unchanged. Programming languages and strategies for it, however, did not. Hardware design or acceleration for certain operations may favour different implementations for higher performance, for example: vectorization, sparse linear algebra; while simplicity may favour for-loops.

Parallel to the topic of wave simulation, the scene of artificial intelligence and machine learning has been growing quickly: In 2015 the Python library Tensorflow was released, followed by PyTorch in 2016, providing easy access to machine learning models through the abstraction and optimization of tensor operations. This trend would only continue over the years. In February 2025 SciPy released its 1.15 version, which included tensor contraction operations for sparse arrays, which opened the opportunity for an easier implementation of a slightly different approach for FDTD.



Typically, numerical solutions of differential equations with the use of sparse linear algebra require the grid space in question to be brought from three or two dimensions to one so it can be operated on with the appropriate function

of the sparse linear algebra library. Now, alternatively, the reshaping of the `array` can be easily skipped without increasing the time complexity of the linear transformations.

The following chapters will present a comprehensive introduction to FDTD and a detailed guide to its implementation using numerical libraries of Python, including Numpy and SciPy.

Chapter 2

Electromagnetic theory and wave propagation

This simulation relies on the classic theory of electromagnetism as compiled and unified by James Clerk Maxwell. The first topic of this thesis shall therefore be the underlying wave-behaviour of vector fields which satisfy the Maxwell Equations.

2.0.1 The Maxwell Equations

$$\nabla \cdot \mathbf{D} = \rho \quad (2.1)$$

$$\nabla \cdot \mathbf{B} = 0 \quad (2.2)$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \quad (2.3)$$

$$\nabla \times \mathbf{H} = \mathbf{J} + \frac{\partial \mathbf{D}}{\partial t} \quad (2.4)$$

The vector fields are implied therein to be functions of both position and time. And for the sake of simplicity of the following proof, we shall assume constant and scalar ε and μ , such that

$$\mathbf{D} = \varepsilon \mathbf{E} \quad (2.5)$$

$$\mathbf{B} = \mu \mathbf{H} \quad (2.6)$$

2.0.2 Ohm's Law

Additionally, Ohm's Law describes the relationship between an electric field and its resulting current density as

$$\mathbf{J} = \sigma \mathbf{E} \quad (2.7)$$

2.1 The electromagnetic wave equations

For a vector field $\mathbf{F}$, the curl of its curl is equal to

$$\nabla \times (\nabla \times \mathbf{F}) = \nabla(\nabla \cdot \mathbf{F}) - \nabla^2 \mathbf{F} \quad (2.8)$$

where ∇^2 is the vector Laplacian operator.

2.1.1 Homogeneous equations

By direct application and substitution onto 2.3 of 2.8 and 2.6, respectively, we obtain the following:

$$\nabla \times (\nabla \times \mathbf{E}) = \nabla(\nabla \cdot \mathbf{E}) - \nabla^2 \mathbf{E} = -\mu \frac{\partial}{\partial t} (\nabla \times \mathbf{H}) \quad (2.9)$$

Schwarz's theorem has been implicitly invoked to switch the curl-operator and time-derivative. Furthermore, since we are studying the propagation capability of electromagnetic fields, we shall consider a situation where no free charges are present to locally excite electric fields, thus yielding $\nabla \cdot \mathbf{D} = 0$, whereas no such assumption is necessary for currents since electric fields and currents are already related as per Ohm's Law. Under these assumptions and the substitution of 2.4 thereon, the wave equation or, rather, an attenuated wave equation is directly obtained.

$$\nabla^2 \mathbf{E} - \mu\sigma \frac{\partial \mathbf{E}}{\partial t} - \mu\varepsilon \frac{\partial^2 \mathbf{E}}{\partial t^2} = 0 \quad (2.10)$$

An equivalent result can be reached for magnetic waves by following the aforementioned steps, now for Ampere-Maxwell's Law (eq. 2.4), thus directly showing that electromagnetic fields naturally exhibit wavelike behaviour and are therefore capable of self-propagation solely by mutual induction.

2.1.2 Non-homogeneous equations

We shall now abstain from making presuppositions regarding the absence of free charges. In this case $\nabla \cdot \mathbf{E} \neq 0$, so applying 2.8 onto 2.3 becomes

$$\nabla \times (\nabla \times \mathbf{E}) = \nabla(\nabla \cdot \mathbf{E}) - \nabla^2 \mathbf{E} = -\mu \frac{\partial}{\partial t} (\nabla \times \mathbf{H}) \quad (2.11)$$

Further resulting in a non-homogeneous electric wave equation

$$\nabla^2 \mathbf{E} - \mu\sigma \frac{\partial \mathbf{E}}{\partial t} - \mu\varepsilon \frac{\partial^2 \mathbf{E}}{\partial t^2} = \nabla(\nabla \cdot \mathbf{E}) \quad (2.12)$$

The derivation of the magnetic equation is slightly different since, contrary to the electric field, $\nabla \cdot \mathbf{H} = 0$. Once again Ohm's Law (eq. 2.7) is substituted in, which, perhaps somewhat surprisingly, yields a homogeneous wave equation:

$$\begin{aligned} \nabla \times (\nabla \times \mathbf{H}) &= \nabla(\nabla \cdot \mathbf{H}) - \nabla^2 \mathbf{H} = \sigma(\nabla \times \mathbf{E}) + \varepsilon \frac{\partial}{\partial t}(\nabla \times \mathbf{E}) \\ \nabla^2 \mathbf{H} - \mu\sigma \frac{\partial \mathbf{H}}{\partial t} - \mu\varepsilon \frac{\partial^2 \mathbf{H}}{\partial t^2} &= 0 \end{aligned} \quad (2.13)$$

At a first glance, one might assume an incoherence between these equations and its possible solutions. Another question might arise: If antennas use currents to generate waves, why does the current density $\mathbf{J}$ not appear in these terms? These intrusive doubts will be dispelled in due time (Section 2.2), although not before new ones are induced.

Solutions to the non-homogeneous wave equation

Not different from the solution of any linear operator, the solution to any non-homogeneous wave equation is the superposition of a particular solution and the solution to the homogenous equation. Consider the following non-homogeneous equation:

$$\left(\nabla^2 - \alpha \frac{\partial}{\partial t} - \beta \frac{\partial^2}{\partial t^2} \right) \Psi(\mathbf{r}, t) = \xi(\mathbf{r}, t)$$

If $\varphi(\mathbf{r}, t)$ is a solution to the non-homogeneous equation and $\mathbf{h}(\mathbf{r}, t)$ is a solution to the homogeneous equation, $\varphi(\mathbf{r}, t) + \mathbf{h}(\mathbf{r}, t)$ is also a solution to the wave equation, because

$$\left(\nabla^2 - \alpha \frac{\partial}{\partial t} - \beta \frac{\partial^2}{\partial t^2} \right) (\varphi(\mathbf{r}, t) + \mathbf{h}(\mathbf{r}, t)) = \xi(\mathbf{r}, t)$$

$$\left(\nabla^2 - \alpha \frac{\partial}{\partial t} - \beta \frac{\partial^2}{\partial t^2} \right) \varphi(\mathbf{r}, t) + \left(\nabla^2 - \alpha \frac{\partial}{\partial t} - \beta \frac{\partial^2}{\partial t^2} \right) \mathbf{h}(\mathbf{r}, t) = \xi(\mathbf{r}, t)$$

$$\left(\nabla^2 - \alpha \frac{\partial}{\partial t} - \beta \frac{\partial^2}{\partial t^2} \right) \varphi(\mathbf{r}, t) + 0 = \xi(\mathbf{r}, t)$$

2.2 Propagation from sources

The remaining question now concerns the propagation of electromagnetic waves originating from a source in a finite region of space to otherwise field-free space. For this purpose, we define a region $\mathcal{V} \subset \mathbb{R}^3$, where, following the

non-homogenous wave equation, $\nabla(\nabla \cdot \mathbf{E}) := \mathbf{G}(\mathbf{r}, t)$ for some arbitrary vector-valued function $G : \mathcal{V} \times \mathbb{R} \rightarrow \mathbb{R}^3 : (\mathbf{r}, t) \mapsto \mathbf{G}(\mathbf{r}, t)$. We further define a **boundary condition** $\mathbf{E}(\mathbf{r}, 0) = \mathbf{H}(\mathbf{r}, 0) = \mathbf{0}$ for $\mathbf{r} \notin \mathcal{V}$. **Therefrom** follows:

$$\nabla^2 \mathbf{E} - \mu\sigma \frac{\partial \mathbf{E}}{\partial t} - \mu\varepsilon \frac{\partial^2 \mathbf{E}}{\partial t^2} = \mathbf{G}(\mathbf{r}, t) : \mathbf{r} \in \mathcal{V} \quad (2.14)$$

$$\nabla^2 \mathbf{E} - \mu\sigma \frac{\partial \mathbf{E}}{\partial t} - \mu\varepsilon \frac{\partial^2 \mathbf{E}}{\partial t^2} = \nabla(\nabla \cdot \mathbf{E}) : \mathbf{r} \notin \mathcal{V} \quad (2.15)$$

Note that there is no pre-emptive assumption that the electric field outside of the region $\mathcal{V}$ is governed by a homogeneous wave equation, because we have not yet studied whether or not the wave propagation conserves the divergence of the field. The answer is however: yes, it does. Even though it might not be obvious by staring at these equations, consider the fact that for any vector field $\mathbf{F}$: $\nabla \cdot (\nabla \times \mathbf{F}) = 0$. Thus, directly from Maxwell's Equations follows:

$$\nabla \cdot (\nabla \times \mathbf{E}) = -\frac{\partial}{\partial t} \nabla \cdot \mathbf{B} = 0 \quad (2.16)$$

$$\nabla \cdot (\nabla \times \mathbf{H}) = \nabla \cdot \mathbf{J} + \frac{\partial}{\partial t} \nabla \cdot \mathbf{D} = 0 \quad (2.17)$$

Hence,

$$\frac{\partial}{\partial t} \nabla \cdot \mathbf{H} = 0 \quad (2.18)$$

$$\frac{\partial}{\partial t} \nabla \cdot \mathbf{E} = -\frac{\sigma}{\varepsilon} (\nabla \cdot \mathbf{E}) \quad (2.19)$$

With the solutions for $\mathbf{r} \notin \mathcal{V}$:

$$(\nabla \cdot \mathbf{H})(\mathbf{r}, t) = 0 \quad (2.20)$$

$$(\nabla \cdot \mathbf{E})(\mathbf{r}, t) = C(\mathbf{r})e^{-\frac{\sigma}{\varepsilon}t} : C(\mathbf{r}) = (\nabla \cdot \mathbf{E})|_{t=0} \quad (2.21)$$

Meaning that the divergence of the electric field in $\mathcal{V}$ is solely determined by the relaxation of charge, which for the aforementioned boundary conditions of the electric and magnetic fields implies that the divergence will remain unchanged outside of the region $\mathcal{V}$.

Now, realistically, $\mathbf{G}$ would also be confined to the same restrictions imposed by the conservation of charge as per eq. 2.17, albeit not by 2.19 because the implication of the assumption of an arbitrary function $\mathbf{G}$ is that arbitrary trajectories are forced upon the charge distribution inside $\mathcal{V}$, hence, charges do not necessarily follow the force imposed by the electric fields.



Avoiding an existential crisis

Furthermore, eq. 2.17 indirectly answers the previous question of "why does $\mathbf{J}$ not appear in the non-homogeneous wave equations when antennas clearly use currents to generate waves?"; $\mathbf{J}$ is indeed implied in the non-homogeneous electric wave equation. But why does the magnetic wave equation remain seemingly homogeneous? It actually does not, because the magnetic wave is inextricably intertwined with the electric wave, thus not every otherwise homogeneous solution to $\mathbf{H}$ results in a valid solution for $\mathbf{E}$.

2.2.1 Necessary equations for the propagation

An important conclusion of this analysis is the realization that for some specific starting conditions; specifically, a field-free space with a controlled source, the propagation can be obtained from the two curl equations, without any additional considerations for the transition between materials of different properties.

Chapter 3

FDTD - Yee's method

As shown in the previous chapter, the propagation of a wave can be **obtained from** the curl of the electric and magnetic fields as described by Maxwell's equations. An already established method for **this purpose** is Yee's Method, also called Finite-Difference Time-Domain (FDTD) Method. This method was introduced by mathematician Kane S. Yee in 1966.

The core of this method consists in **intercalating** the calculations of the electric and magnetic fields in a leapfrog fashion: in the mathematical discretization the fields are considered to exist not only at integer grid points, in both space and time, but also at "half-points" in between the **integers**. The details will be **shown in the derivations of the equations**.

3.1 Finite differences

The first step is to derive an approximation of the derivative using finite differences. This can be easily achieved by considering the Taylor Expansion of an arbitrary function $f(x)$ around a point x_0 , as follows:

$$f(x_0 + \Delta x) = \sum_{n=0}^{\infty} \frac{1}{n!} \frac{d^n f}{dx^n}(x_0) \Delta x^n \quad (3.1)$$

3.1.1 Central differences

Given the nature of Yee's Method, in which half-points between integer grid points are assumed, the ideal discrete derivative option is Central Differences, as opposed to **Forward** or **Backward** Differences. For this purpose, we consider the expansion at $f(x_0)$ with $\Delta x = \pm\delta/2$,

$$f\left(x_0 + \frac{\delta}{2}\right) = f(x_0) + \frac{\delta}{2} \frac{df}{dx}(x_0) + \mathcal{O}(\delta^2) \quad (3.2)$$

$$f\left(x_0 - \frac{\delta}{2}\right) = f(x_0) - \frac{\delta}{2} \frac{df}{dx}(x_0) + \mathcal{O}(\delta^2) \quad (3.3)$$

By subtracting these equations we get as a result,

$$\therefore \frac{df}{dx}(x_0) = \frac{f\left(x_0 + \frac{\delta}{2}\right) - f\left(x_0 - \frac{\delta}{2}\right)}{\delta} + \mathcal{O}(\delta^2) \quad (3.4)$$

3.2 Yee cells

The main structure for FDTD is the "Yee cell". As stated previously, this consists of points and half-points. For the 1-dimensional case, the electric field is calculated on the integer points, and the magnetic field is defined between them. This remains true for both bases orthogonal to the propagation vector. A visual representation thereof can be seen in Figure 3.1, where it can also be easily understood why Central Differences work well for both fields.

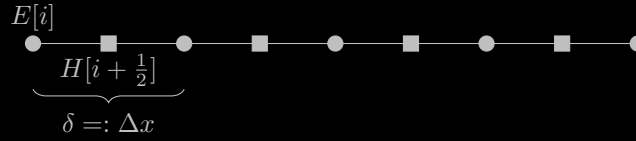


Figure 3.1: Yee cells in one dimension

The 2-Dimensional Yee cell is already well known and defined. However, an argument will nonetheless be presented to justify why the current convention was chosen, with visual representations (Figures 3.2 and 3.3) based closely on Figures 8.1, 8.9 and 8.10 of "*John B.Schneider (2025). Understanding the Finite-Difference Time-Domain Method*".

One might naively define that every integer point shall belong to electric fields and half-points to magnetic fields. This would lead to discrepancies in the Central Differences method for the approximation of the derivative, since field-points would lie diagonally to each other on the grid. A simple way to solve this problem is to consider the TE and TM modes of wave-propagation.

3.2.1 TE-Propagation

To conserve the consistency of the values of Δx and Δy as horizontal and vertical distance between integer points respectively, which are to be used in the Central Differences, the grid for TE-Waves must be defined as per Figure 3.2.

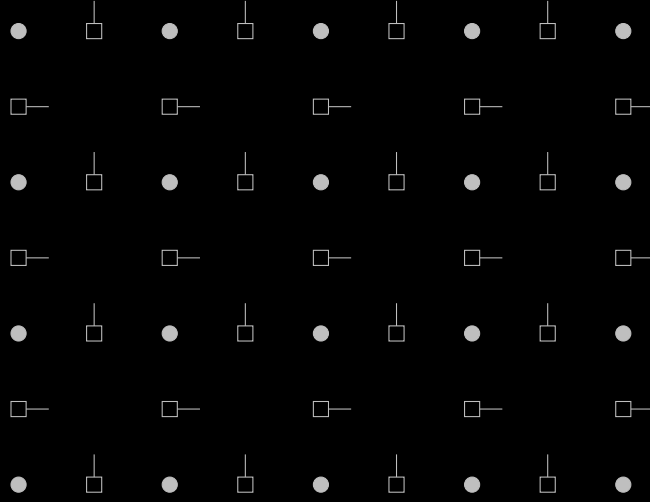


Figure 3.2: 2-Dimensional TE-Yee Cell

An important characteristic in this particular cell as well as of the 1D cell is that: in any direction of propagation, the boundaries are defined to be integer points, thus subsequently leading to the boundary conditions always being defined by the electric field, **because the magnetic fields on the boundary do not propagate outwards.**

3.2.2 TM-Propagation

For the consistency of the aforementioned properties of the previously defined cells, the grid for TM-Waves must be defined following Figure 3.3.

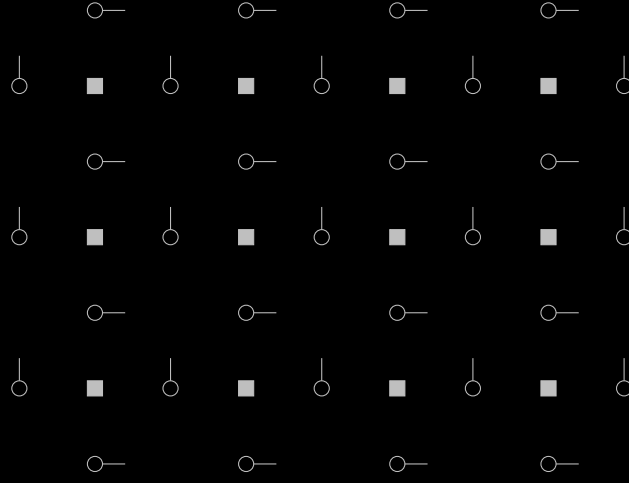


Figure 3.3: 2-Dimensional TM-Yee Cell

3.2.3 2D Yee Cell

With the superposition of the TE and TM grids, the complete 2D Yee Cell is obtained.

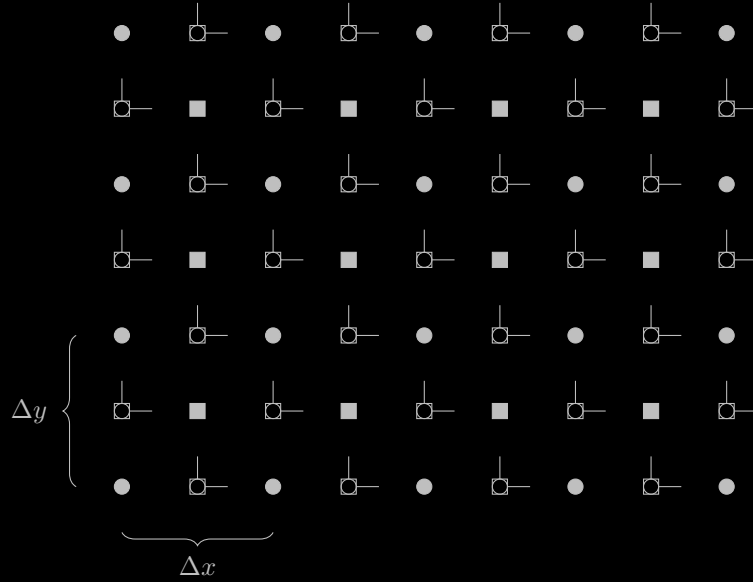


Figure 3.4: 2-Dimensional Yee Cells

Following Figure 3.4, the points at which each field lies can be read:

$$\begin{aligned}
E_z &\mapsto [i, j] \\
E_x &\mapsto \left[i + \frac{1}{2}, j \right] \\
E_y &\mapsto \left[i, j + \frac{1}{2} \right]
\end{aligned} \tag{3.5}$$

where i and j represent the discrete x and y positions respectively.

Lastly, in the time coordinate, electric and magnetic fields are assigned as follows:

$$\begin{aligned}
E &\mapsto [q] \\
H &\mapsto \left[q + \frac{1}{2} \right]
\end{aligned} \tag{3.6}$$

Where q represents the **discrete** time.

This means that the electric and magnetic fields are calculated in an alternating manner.

3.3 Discretization in 2D

With **a** structure and a differentiation method **in** hand, it is now possible to discretize Maxwell's equations **upon** a Yee-Grid. As already mentioned, only Faraday's Law and Ampere-Maxwell's Law are necessary **for these particular starting conditions**.



3.3.1 Faraday's Law

As already presented in eq. 2.3, Faraday's Law states:

$$\nabla \times \mathbf{E} = -\mu \frac{\partial \mathbf{H}}{\partial t}$$

In Cartesian Coordinates this can be written as

$$\therefore \begin{pmatrix} \partial_y E_z - \partial_z E_y \\ \partial_z E_x - \partial_x E_z \\ \partial_x E_y - \partial_y E_x \end{pmatrix} = -\mu \begin{pmatrix} \partial_t H_x \\ \partial_t H_y \\ \partial_t H_z \end{pmatrix} \tag{3.7}$$

Since this simulation is two-dimensional, it means that the fields are not considered to be dependent on the z-axis, which means $\partial_z \rightarrow 0$. This simplifies the equations or, alternatively, yields the following three equations:

$$\partial_y E_z = -\mu \partial_t H_x \quad (3.8)$$

$$\partial_x E_z = \mu \partial_t H_y \quad (3.9)$$

$$\partial_x E_y - \partial_y E_x = -\mu \partial_t H_z \quad (3.10)$$

3.3.2 Ampere-Maxwell

Previously presented in eq. 2.4, Ampere-Maxwell's Law states

$$\nabla \times \mathbf{H} = \sigma \mathbf{E} + \varepsilon \frac{\partial \mathbf{E}}{\partial t}$$

Which in Cartesian Coordinates is equal to

$$\therefore \begin{pmatrix} \partial_y H_z - \partial_z H_y \\ \partial_z H_x - \partial_x H_z \\ \partial_x H_y - \partial_y H_x \end{pmatrix} = \sigma \begin{pmatrix} E_x \\ E_y \\ E_z \end{pmatrix} + \varepsilon \begin{pmatrix} \partial_t E_x \\ \partial_t E_y \\ \partial_t E_z \end{pmatrix} \quad (3.11)$$

similarly, with $\partial_z \rightarrow 0$, the following three equations are obtained:

$$\partial_y H_z = \sigma E_x + \varepsilon \partial_t E_x \quad (3.12)$$

$$-\partial_x H_z = \sigma E_y + \varepsilon \partial_t E_y \quad (3.13)$$

$$\partial_x H_y - \partial_y H_x = \sigma E_z + \varepsilon \partial_t E_z \quad (3.14)$$

3.4 Discretization

The six previously gained equations can be thus discretized utilizing central finite differences to be then applied on a Yee Grid. For convenience, the propagation mode of all six equations (TM or TE) can be easily recognized, and thus be reorganized and regrouped as such.

An example of the discretization method will now be presented to clarify the nuances necessary for the process.

Equation 3.14 must be discretized **as** a point of the form $[i, j, q + 1/2]$, since the spatial derivatives would require $[i \pm 1/2, j, q + 1/2]$ and $[i, j \pm 1/2, q + 1/2]$ **and** the time-derivatives require $[i, j, q + 1]$ and $[i, j, q]$. This leads to almost all values already being defined at the necessary points, with one exception:

$$\begin{aligned} & \varepsilon \frac{1}{\Delta t} \left(E_{z|i|j}^{q+1} - E_{z|i|j}^q \right) + \sigma E_{z|i|j}^{q+\frac{1}{2}} \\ &= \frac{1}{\Delta x} \left(H_{y|i+\frac{1}{2}|j}^{q+\frac{1}{2}} - H_{y|i-\frac{1}{2}|j}^{q+\frac{1}{2}} \right) - \frac{1}{\Delta y} \left(H_{x|i|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{x|i|j-\frac{1}{2}}^{q+\frac{1}{2}} \right) \end{aligned} \quad (3.15)$$

Here $\sigma E_{z|i|j}^{q+\frac{1}{2}}$ is not defined, because electric fields are only defined at integer points. The solution for this is to build the average value of $[i, j, q + 1]$ and $[i, j, q]$. This step can already be appreciated in the solutions that are presented next.

3.4.1 TE Waves

Equations 3.8, 3.9, 3.14 describe the propagation of TE waves, and are discretized as follows:

$$\mu \frac{1}{\Delta t} \left(H_{z|i|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{z|i|j-\frac{1}{2}}^{q+\frac{1}{2}} \right) = \frac{1}{\Delta y} \left(E_{z|i|j+1}^q - E_{z|i|j}^q \right) \quad (3.16)$$

$$\mu \frac{1}{\Delta t} \left(H_{y|i+\frac{1}{2}|j}^{q+\frac{1}{2}} - H_{y|i-\frac{1}{2}|j}^{q+\frac{1}{2}} \right) = \frac{1}{\Delta x} \left(E_{z|i+1|j}^q - E_{z|i|j}^q \right) \quad (3.17)$$

$$\begin{aligned} & \varepsilon \frac{1}{\Delta t} \left(E_{z|i|j}^{q+1} - E_{z|i|j}^q \right) + \sigma \frac{1}{2} \left(E_{z|i|j}^{q+1} + E_{z|i|j}^q \right) \\ &= \frac{1}{\Delta x} \left(H_{y|i+\frac{1}{2}|j}^{q+\frac{1}{2}} - H_{y|i-\frac{1}{2}|j}^{q+\frac{1}{2}} \right) - \frac{1}{\Delta y} \left(H_{x|i|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{x|i|j-\frac{1}{2}}^{q+\frac{1}{2}} \right) \end{aligned} \quad (3.18)$$

3.4.2 TM Waves

Equations 3.10, 3.12, 3.13 describe the TM waves, and are discretized as follows:

$$\begin{aligned} & \mu \frac{1}{\Delta t} \left(H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{z|i+\frac{1}{2}|j-\frac{1}{2}}^{q+\frac{1}{2}} \right) \\ &= \frac{1}{\Delta x} \left(E_{y|i+1|j+\frac{1}{2}}^q - E_{y|i|j+\frac{1}{2}}^q \right) - \frac{1}{\Delta y} \left(E_{x|i+\frac{1}{2}|j+1}^q - E_{x|i+\frac{1}{2}|j}^q \right) \end{aligned} \quad (3.19)$$

$$\begin{aligned} & \varepsilon \frac{1}{\Delta t} \left(E_{x|i+\frac{1}{2}|j}^{q+1} - E_{x|i+\frac{1}{2}|j}^q \right) + \sigma \frac{1}{2} \left(E_{x|i+\frac{1}{2}|j}^{q+1} + E_{x|i+\frac{1}{2}|j}^q \right) \\ &= \frac{1}{\Delta y} \left(H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{z|i+\frac{1}{2}|j-\frac{1}{2}}^{q+\frac{1}{2}} \right) \end{aligned} \quad (3.20)$$

$$\begin{aligned}
\varepsilon \frac{1}{\Delta t} \left(E_{y|i|j+\frac{1}{2}}^{q+\frac{1}{2}} - E_{y|i|j+\frac{1}{2}}^{q-\frac{1}{2}} \right) + \sigma \frac{1}{2} \left(E_{y|i|j+\frac{1}{2}}^{q+\frac{1}{2}} + E_{y|i|j+\frac{1}{2}}^{q-\frac{1}{2}} \right) \\
= -\frac{1}{\Delta x} \left(H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{z|i-\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} \right)
\end{aligned} \tag{3.21}$$

3.4.3 Update Equations

The objective is now to find the relationship between past and future values for each field, **with which they can be "updated"** during the simulation. Each of the six equations already presents those relationships and just require some algebraic manipulation to yield in explicit form:

TE Waves

$$H_{x|i|j+\frac{1}{2}}^{q+\frac{1}{2}} = H_{x|i|j+\frac{1}{2}}^{q-\frac{1}{2}} - \frac{\Delta t}{\mu \Delta y} \left(E_{z|i|j+1}^q - E_{z|i|j}^q \right) \tag{3.22}$$

$$H_{y|i+\frac{1}{2}|j}^{q+\frac{1}{2}} = H_{y|i+\frac{1}{2}|j}^{q-\frac{1}{2}} + \frac{\Delta t}{\mu \Delta x} \left(E_{z|i+1|j}^q - E_{z|i|j}^q \right) \tag{3.23}$$

$$\begin{aligned}
E_{z|i|j}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{z|i|j}^q \\
&+ \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \left[\frac{\Delta t}{\varepsilon \Delta x} \left(H_{y|i+\frac{1}{2}|j}^{q+\frac{1}{2}} - H_{y|i-\frac{1}{2}|j}^{q+\frac{1}{2}} \right) - \frac{\Delta t}{\varepsilon \Delta y} \left(H_{x|i|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{x|i|j-\frac{1}{2}}^{q+\frac{1}{2}} \right) \right]
\end{aligned} \tag{3.24}$$

TM Waves

$$\begin{aligned}
H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} &= H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q-\frac{1}{2}} \\
&- \frac{\Delta t}{\mu \Delta x} \left(E_{y|i+1|j+\frac{1}{2}}^q - E_{y|i|j+\frac{1}{2}}^q \right) + \frac{\Delta t}{\mu \Delta y} \left(E_{x|i+\frac{1}{2}|j+1}^q - E_{x|i+\frac{1}{2}|j}^q \right)
\end{aligned} \tag{3.25}$$

$$\begin{aligned}
E_{x|i+\frac{1}{2}|j}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{x|i+\frac{1}{2}|j}^q \\
&+ \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \frac{\Delta t}{\varepsilon \Delta y} \left(H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{z|i+\frac{1}{2}|j-\frac{1}{2}}^{q+\frac{1}{2}} \right)
\end{aligned} \tag{3.26}$$

$$\begin{aligned}
E_{y|i|j+\frac{1}{2}}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{y|i|j+\frac{1}{2}}^q \\
&- \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \frac{\Delta t}{\varepsilon \Delta x} \left(H_{z|i+\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} - H_{z|i-\frac{1}{2}|j+\frac{1}{2}}^{q+\frac{1}{2}} \right)
\end{aligned} \tag{3.27}$$

Chapter 4

Implementation through tensors

The implementation of this simulator will be in Python and, to avoid slow for-loops while at the same time taking advantage of diverse libraries like Numpy, SciPy, etc., an alternative representation of the field components and operations **thereon** will be necessary. One such method may be with linear algebra, mapping the field **in an array of the form** $F_{ij} \in \mathbb{R}^M \times \mathbb{R}^N$ to a vector of the form $F_{n=i+Nj} \in \mathbb{R}^{M+N}$ and find a matrix to represent the linear combinations of the neighbouring components. However, by the title of this section, one might correctly guess that this will not be our case.

Alternatively, the fields can be left in their original form of F_{ij} , **whilst** the linear combinations of the neighbouring components **are** be represented by tensor **products**. This is the method that will be used for the implementation. The challenge is to find the operating tensors. **For a more mathematical explanation justify why the presented method works, check Appendix A.**

Firstly, it is important to note that arrays in programming languages are usually indexed exclusively by integer values. Therefore, every field defined **in** half-points must be translated to contain only integer indices. This results in equations that cannot be interpreted geometrically quite as **good** as before, however, by operating on them the same way, the end result does not change, and it opens the possibility to more easily interpret the fields as 2D-arrays.

4.1 Updated Update Equations

The result is a similar set of equations that now easily fit in our interpretation:

Field	Fraction index	Whole index
E	$n + \frac{1}{2}$	n
E	$n - \frac{1}{2}$	$n - 1$
H	$n + \frac{1}{2}$	n
H	$n - \frac{1}{2}$	$n - 1$



Table 4.1: Index matching

TE Waves

$$H_{x|i|j}^q = H_{x|i|j}^{q-1} - \frac{\Delta t}{\mu \Delta y} \left(E_{z|i|j+1}^q - E_{z|i|j}^q \right) \quad (4.1)$$

$$H_{y|i|j}^q = H_{y|i|j}^{q-1} + \frac{\Delta t}{\mu \Delta x} \left(E_{z|i+1|j}^q - E_{z|i|j}^q \right) \quad (4.2)$$

$$\begin{aligned} E_{z|i|j}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{z|i|j}^q \\ &+ \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \left[\frac{\Delta t}{\varepsilon \Delta x} \left(H_{y|i|j}^q - H_{y|i-1|j}^q \right) - \frac{\Delta t}{\varepsilon \Delta y} \left(H_{x|i|j}^q - H_{x|i|j-1}^q \right) \right] \end{aligned} \quad (4.3)$$

TM Waves

$$\begin{aligned} H_{z|i|j}^q &= H_{z|i|j}^{q-1} \\ &- \frac{\Delta t}{\mu \Delta x} \left(E_{y|i+1|j}^q - E_{y|i|j}^q \right) + \frac{\Delta t}{\mu \Delta y} \left(E_{x|i|j+1}^q - E_{x|i|j}^q \right) \end{aligned} \quad (4.4)$$

$$\begin{aligned} E_{x|i|j}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{x|i|j}^q \\ &+ \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \frac{\Delta t}{\varepsilon \Delta y} \left(H_{z|i|j}^q - H_{z|i|j-1}^q \right) \end{aligned} \quad (4.5)$$

$$\begin{aligned} E_{y|i|j}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{y|i|j}^q \\ &- \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \frac{\Delta t}{\varepsilon \Delta x} \left(H_{z|i|j}^q - H_{z|i-1|j}^q \right) \end{aligned} \quad (4.6)$$

4.2 Fields and operators

For a faster and performance-friendly simulation, only TE-waves will be implemented. The following methods are, however, equally valid for TM-waves.

As per Figure 3.4, it can be easily observed that TE-waves are what determine the total size of the simulation space. Further inspection leads to the necessary size of every field

$$E_z \in \mathbb{R}^{M \times N} \wedge E_x \in \mathbb{R}^{M \times N-1} \wedge E_y \in \mathbb{R}^{M-1 \times N} \quad (4.7)$$

$$H_z \in \mathbb{R}^{M-1 \times N-1} \wedge H_x \in \mathbb{R}^{M-1 \times N} \wedge H_y \in \mathbb{R}^{M \times N-1} \quad (4.8)$$

One detail of extreme importance and easy to ignore is that, both in mathematics and programming, arrays are defined as $A_{\text{row}, \text{column}}$, thus A_{ij} would index A_{yx} and not A_{xy} , if no further action is taken. To avoid this and to use the more intuitive approach, all arrays will be considered to be initially transposed. This means that M and N determine the Y and X array lengths respectively.

With the purpose of decluttering the subsequent expressions, Einstein's Summation Convention will be henceforth used. A matrix-vector multiplication is written by omitting the summation symbol and directly treating a repeating index as a summation index as follows:

$$w_i = A_{ik}v_k \equiv \sum_k A_{ik}v_k \quad (4.9)$$

4.2.1 Code so far

It is now possible to start programming the simulation. For ease of use and reasons that will become apparent later, the simulator will be wrapped in a class.

```

1 import numpy as np
2
3 class FDTD_EM(object):
4     def __init__(parameters go here):
5         #PARAMETERS
6         self.SIZE_X = size_x
7         self.SIZE_Y = size_y
8         self.DELTA_X = delta_x
9         self.DELTA_Y = delta_y
10        self.DELTA_T = delta_t
11        self.time = 0
12        #FIELDS
13        self.Ez = np.zeros((self.SIZE_X, self.SIZE_Y))
14        self.Ex = np.zeros((self.SIZE_X - 1, self.SIZE_Y))
15        self.Ey = np.zeros((self.SIZE_X, self.SIZE_Y - 1))
16        self.Hz = np.zeros((self.SIZE_X - 1, self.SIZE_Y - 1))
17        self.Hx = np.zeros((self.SIZE_X, self.SIZE_Y - 1))
18        self.Hy = np.zeros((self.SIZE_X - 1, self.SIZE_Y))

```



4.2.2 Operators for $H_{x|i|j}^q$

$$\equiv H_{x|i|j}^q = H_{x|i|j}^{q-1} - \frac{\Delta t}{\mu \Delta y} \left(E_{z|i|j+1}^q - E_{z|i|j}^q \right) \quad (4.10)$$

The linear operation takes place on the j^{th} axis, thus an expression can be derived from the following form

$$\underbrace{\begin{pmatrix} H_{x|i|1} \\ H_{x|i|2} \\ \vdots \\ H_{x|i|M-1} \end{pmatrix}}_{\mathbb{R}^{M-1}} \sim \underbrace{\begin{pmatrix} -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 & 1 \end{pmatrix}}_{\mathbb{R}^{M-1} \times M} \underbrace{\begin{pmatrix} E_{z|i|1} \\ E_{z|i|2} \\ \vdots \\ E_{z|i|M} \end{pmatrix}}_{\mathbb{R}^M} \quad (4.11) \quad \equiv$$

which by inspection yields

$$H_{x|ij}^q = H_{x|ij}^{q-1} - \frac{\Delta t}{\mu \Delta y} E_{z|ik}^q F_{jk} \quad (4.12)$$

Or, equivalently

$$H_{x|ij}^q = H_{x|ij}^{q-1} - \frac{\Delta t}{\mu \Delta y} F_{ik} E_{z|jk}^q \quad (4.13)$$

with

$$F = \begin{pmatrix} -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 & 1 \end{pmatrix} \in \mathbb{R}^{M-1 \times M} \quad (4.14)$$

Where we assign the matrix the name "forward matrix" (hence the symbol F), **since** for each index n it operates with the following index $n + 1$.

4.2.3 Operator for $H_{y|i|j}^q$

$$H_{y|i|j}^q = H_{y|i|j}^{q-1} + \frac{\Delta t}{\mu \Delta x} \left(E_{z|i+1|j}^q - E_{z|i|j}^q \right) \quad (4.15)$$

The linear operation takes place on the i^{th} axis, thus an expression can be derived from the following form:

$$\underbrace{\begin{pmatrix} H_{x|1|j} \\ H_{x|2|j} \\ \vdots \\ H_{x|N-1|j} \end{pmatrix}}_{\mathbb{R}^{N-1}} \sim \underbrace{\begin{pmatrix} -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 & 1 \end{pmatrix}}_{\mathbb{R}^{N-1 \times N}} \underbrace{\begin{pmatrix} E_{z|1|j} \\ E_{z|2|j} \\ \vdots \\ E_{z|N|j} \end{pmatrix}}_{\mathbb{R}^N} \quad (4.16)$$

which, once more, by inspection yields

$$H_{y|i|j}^q = H_{y|i|j}^{q-1} + \frac{\Delta t}{\mu \Delta x} F_{ik} E_{z|k|j}^q \quad (4.17)$$

with another forward matrix, now of different dimensions,

$$F = \begin{pmatrix} -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 & 1 \end{pmatrix} \in \mathbb{R}^{M-1 \times M} \quad (4.18)$$

4.2.4 Operator for $E_{z|i|j}^q$

$$\begin{aligned} E_{z|i|j}^{q+1} &= \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{z|i|j}^q \\ &+ \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \left[\frac{\Delta t}{\varepsilon \Delta x} \left(H_{y|i|j}^q - H_{y|i-1|j}^q \right) - \frac{\Delta t}{\varepsilon \Delta y} \left(H_{x|i|j}^q - H_{x|i|j-1}^q \right) \right] \end{aligned} \quad (4.19)$$

This case requires a linear combination of two operations. **Firstly,**

$$\underbrace{\begin{pmatrix} E_{z|1|j} \\ E_{z|2|j} \\ E_{z|3|j} \\ \vdots \\ E_{z|N|j} \end{pmatrix}}_{\mathbb{R}^N} \sim \underbrace{\begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix}}_{\mathbb{R}^{N \times N-1}} \underbrace{\begin{pmatrix} H_{y|1|j} \\ H_{y|2|j} \\ H_{y|3|j} \\ \vdots \\ H_{z|N-1|j} \end{pmatrix}}_{\mathbb{R}^{N-1}} \quad (4.20)$$

and

$$\underbrace{\begin{pmatrix} E_{z|i|1} \\ E_{z|i|2} \\ E_{z|i|3} \\ \vdots \\ E_{z|i|M} \end{pmatrix}}_{\mathbb{R}^M} \sim \underbrace{\begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix}}_{\mathbb{R}^{M \times M-1}} \underbrace{\begin{pmatrix} H_{x|i|1} \\ H_{x|i|2} \\ H_{x|i|3} \\ \vdots \\ H_{z|i|M-1} \end{pmatrix}}_{\mathbb{R}^{M-1}} \quad (4.21)$$

This yields a similar result as for the previous fields,

$$E_{z|ij}^{q+1} = \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{z|ij}^q + \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \left[\frac{\Delta t}{\varepsilon \Delta x} B_{ik} H_{y|kj}^q - \frac{\Delta t}{\varepsilon \Delta y} H_{x|ik}^q B_{jk} \right] \quad (4.22)$$

Equivalently,

$$E_{z|ij}^{q+1} = \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} E_{z|ij}^q + \frac{1}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \left[\frac{\Delta t}{\varepsilon \Delta x} B_{ik} H_{y|kj}^q - \frac{\Delta t}{\varepsilon \Delta y} B_{ik} H_{x|jk}^q \right] \quad (4.23)$$

now with matrices of a different format, to be **henceforth** referred to as "backward matrix":

$$B = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix} \in \mathbb{R}^{N \times N-1} \vee \in \mathbb{R}^{M \times M-1} \quad (4.24)$$

Important notes:

- To avoid cumbersome notation, forward and backward matrices are simply referred to as F and B respectively, and are simply considered to adapt their dimensions to the tensor they are operating on. In the implementation, these matrices can easily be defined with different sizes to fit our needs, with two simple functions.
- The backward matrices for E_z are defined to result in a field-free boundary, thus acting as a perfect reflector. This can be simply overwritten afterwards, however, not easily nor conveniently through tensors, to obtain absorbing boundary conditions.

4.2.5 Code so far

The two types of matrices can be easily obtained with the functions implemented in the following code. For each field, matrices adjusted to their particular dimensions must be defined. They are then added to a member variable of the class:

```

1 import numpy as np
2
3 class FDTD_EM(object):
4     def __init__(parameters go here):
5         ... parameters
6         ... fields
7         #TENSOR OPERATORS
8         self.Hx_Ez
9         self.Hy_Ez
10        self.Ez_Hy
11        self.Ez_Hx
12
13    def __forwards_matrix(self, rows, cols):
14        F = np.zeros((rows, cols))
15        index = min(rows, cols)
16        for i in range(index):
17            F[i, i] = -1
18            if (i < cols-1):
19                F[i, i + 1] = 1
20        return F
21
22    def __backwards_matrix(self, rows, cols):
23        B = np.zeros((rows, cols))
24        index = min(rows, cols)
25        for i in range(index):
26            B[i, i] = 1
27            if (i < rows - 1):
28                B[i + 1, i] = -1
29
30        B[0, 0] = 0
31        B[-1, -1] = 0
32        return B

```

4.3 The constants and materials

The last pieces of the equations still patiently await their discretization: ε , μ , and σ . As already mentioned, this work will focus on studying the electric interactions of electromagnetic waves, as it is what mainly determines their behaviour. The magnetic permeability will therefore be considered to be the universal constant μ_0 throughout the simulation space.

The permittivity ε and conductivity σ must be defined everywhere there is a field, meaning they must span $\mathbb{R}^{M \times N}$. This tensor, or rather, a 2D-array is used in an element-wise multiplication. In the case of TE-waves, this fits perfectly the size of the only electric field component E_z . For TM-waves these arrays can simply be sliced or broadcasted to the other, smaller arrays of E_x and E_y .

Their half-point jurisdictions are derived directly from Table 4.1, thus, $[i, j + 1/2]$, $[i + 1/2, j]$ and $[i + 1/2, j + 1/2]$ fall within the space of ε_{ij} .

4.3.1 Propagation constants

With ε_{ij} and σ_{ij} already defined, to avoid both unnecessary calculations during **the execution of the program**, and mathematical clutter, the following constants are defined within the three (or six, **if further implemented**) update equations:

$$(C_{EzE})_{ij} = \frac{1 - \frac{\sigma_{ij} \Delta t}{2\varepsilon_{ij}}}{1 + \frac{\sigma_{ij} \Delta t}{2\varepsilon_{ij}}} \quad (4.25)$$

$$(C_{EzH})_{ij} = \frac{1}{\varepsilon_{ij} \left(1 + \frac{\sigma_{ij} \Delta t}{2\varepsilon_{ij}}\right)} \quad (4.26)$$

Whereas C_{ExE} , C_{ExH} , C_{EyE} , C_{EyH} would be similarly defined, with only their dimensions adapted to their respective fields.

4.4 Final symbolic formulation

With the operations and constants defined, a mathematical form which is compact and can be directly written in Python, can now be **gained**:

$$H_{x|ij}^q = H_{x|ij}^{q-1} - \frac{1}{\mu} \odot E_{z|ik}^q \left(F_{jk} \frac{\Delta t}{\Delta y} \right) \quad (4.27)$$

$$H_{y|ij}^q = H_{y|ij}^{q-1} + \frac{1}{\mu} \odot \left(F_{ik} \frac{\Delta t}{\Delta x} \right) E_{z|kj}^q \quad (4.28)$$

$$\begin{aligned} E_{z|ij}^{q+1} &= (C_{EzE})_{ij} \odot E_{z|ij}^q \\ &+ (C_{EzH})_{ij} \odot \left[\left(B_{ik} \frac{\Delta t}{\Delta x} \right) H_{y|kj}^q - H_{x|ik}^q \left(B_{jk} \frac{\Delta t}{\Delta y} \right) \right] \end{aligned} \quad (4.29)$$

Or,

$$H_{x|ij}^q = H_{x|ij}^{q-1} - \frac{1}{\mu} \odot \left(F_{ik} \frac{\Delta t}{\Delta y} \right) E_{z|jk}^q \quad (4.30)$$

$$H_{y|ij}^q = H_{y|ij}^{q-1} + \frac{1}{\mu} \odot \left(F_{ik} \frac{\Delta t}{\Delta x} \right) E_{z|kj}^q \quad (4.31)$$

$$\begin{aligned} E_{z|ij}^{q+1} &= (C_{EzE})_{ij} \odot E_{z|ij}^q \\ &+ (C_{EzH})_{ij} \odot \left[\left(B_{ik} \frac{\Delta t}{\Delta x} \right) H_{y|kj}^q - \left(B_{ik} \frac{\Delta t}{\Delta y} \right) H_{x|jk}^q \right] \end{aligned} \quad (4.32)$$

where:

- $\odot$ represents the Hadamard, Schur, or element-wise product, or, a **plain scalar multiplication in the case of $1/\mu$** . It is left in that notation as a reminder that μ might as well be defined as an array.
- The tensor operators are already scaled with $\Delta t/\Delta r$ to reduce the number of calculations during the main loop.

4.4.1 Code so far

With the final formulation now defined, it is possible to build a **sturdy scaffold** for the propagation mechanics. To greatly improve performance, the tensor operators are defined as **sparse, with the help of SciPy**.

```

1 import numpy as np
2 import scipy as sp
3
4 class FDTD_EM(object):
5
6     MU_0 = sp.constants.mu_0
7     EPSILON_0 = sp.constants.epsilon_0
8
9     def __init__(parameters go here):
10         ... parameters
11         ... fields
12         #TENSOR OPERATORS
13         self.Hx_Ez = sp.sparse.coo_array
14         (
15             self.__forwards_matrix(self.SIZE_Y - 1, self.SIZE_Y)
16             * self.DELTA_T / self.DELTA_Y
17         )
18         self.Hy_Ez = sp.sparse.coo_array
19         (
20             self.__forwards_matrix(self.SIZE_X - 1, self.SIZE_X)
21             * self.DELTA_T / self.DELTA_X
22         )
23         self.Ez_Hy = sp.sparse.coo_array
24         (
25             self.__backwards_matrix(self.SIZE_X, self.SIZE_X - 1)
26             * self.DELTA_T / self.DELTA_X
27         )
28         self.Ez_Hx = sp.sparse.coo_array
29         (
30             self.__backwards_matrix(self.SIZE_Y, self.SIZE_Y - 1)
31             * self.DELTA_T / self.DELTA_Y
32         )
33
34         #MATERIALS
35         self.epsilon = np.ones_like(self.Ez) * self.EPSILON_0
36         self.conductivity = np.zeros_like(self.Ez)
37         self.material_patches = [] #For matplotlib purposes
38         # PROPAGATION CONSTANTS
39         self.C_EzE = (1 - (self.conductivity * self.DELTA_T)
40                       / (2 * self.epsilon))
41                       / (1 + (self.conductivity * self.DELTA_T)

```

```

42         / (2 * self.epsilon))
43     self.C_EzH = 1
44         / (self.epsilon *
45         (1 + (self.conductivity * self.DELTA_T)
46         / (2 * self.epsilon)))
47
48     def __generate_constants(self): ...
49
50     def __forwards_matrix(self, rows, cols): ...
51     def __backwards_matrix(self, rows, cols): ...

```

The material properties can be easily changed with functions. The propagation constants must be updated with every change, thus a function for that purpose is added as well.

```

1  import numpy as np
2  import scipy as sp
3
4  class FTDT_EM(object):
5
6      ... universal constants
7
8      def __init__(parameters go here):
9          ... parameters
10         ... fields
11         ... tensors
12         ... materials
13         ... propagation constants
14
15     def __forwards_matrix(self, rows, cols): ...
16     def __backwards_matrix(self, rows, cols): ...
17
18     def __generate_constants(self): ...
19
20     def add_material(parameters go here): ...

```



Last but not least, the iteration to update the fields

```

1  import numpy as np
2  import scipy as sp
3
4  class FTDT_EM(object):
5
6      ... universal constants
7
8      def __init__(parameters go here):
9          ... parameters
10         ... fields
11         ... tensors
12         ... materials
13         ... propagation constants
14
15     def __forwards_matrix(self, rows, cols): ...
16     def __backwards_matrix(self, rows, cols): ...
17     def __generate_constants(self): ...

```

```

18     def add_material(parameters go here): ...
19
20     def update(self):
21         self.Hx = self.Hx
22             - self.Hx_Ez.tensordot(self.Ez, axes=([1], [1])).T
23             * (1 / self.MU_0)
24         self.Hy = self.Hy
25             + self.Hy_Ez.tensordot(self.Ez, axes=([1], [0]))
26             * (1 / self.MU_0)
27         self.Ez = self.C_EzE * self.Ez
28             + self.C_EzH *
29             (self.Ez_Hy.tensordot(self.Hy, axes=([1], [0]))
30             - self.Ez_Hx.tensordot(self.Hx, axes=([1], [1])).T)

```

Note that in SciPy, tensor multiplication is a member method of the sparse array class, rather than a static function. `A.tensordot(B, ...)`, where A is a sparse array and B is not, yields a non-sparse array as a result, and it conserves the order of the indices; so $A_{jk}B_{ki}$ yields C_{ji} . For that reason, some results must be transposed. The rest of the code, including sources, will be added once the propagation and boundary conditions are completed.

Chapter 5

Mur's Absorbing Boundary Condition

Although the simulation so far is capable of propagation, waves, by design of the operators, will perfectly reflect from the boundaries, instead of being absorbed by them, as if the waves just propagate outwards from the simulation space. The approach for this thesis's absorbing boundary conditions is that of Gerrit Mur (Mur's ABC), as presented in his work, *"Absorbing Boundary Conditions for the Finite-Difference Approximation of the Time-Domain Electromagnetic-Field Equations."*, with a formulation closely based on G. Zheng, A. A. Kishk, A. W. Glisson, and A. B. Yakovlev, *"Implementation of Mur's absorbing boundaries with periodic structures to speed up the design process using Finite-Difference Time-Domain Method, University of Mississippi, 2006."*

5.1 Derivation of Mur's ABC

Let there be a wave source limited to a finite region of space $\Omega \subset \mathbb{R}^3$, whose waves are allowed to propagate infinitely far. Obviously, for all points $\mathbf{r} \in \partial\Omega$ follows that any wave $\varphi(\mathbf{r})$ thereon is a solution to the wave equation. Because the waves are allowed to travel infinitely far, thus avoiding reflections, and the general wave equation allows propagation in any direction, the claim is that there exists a more limiting equation or operator that would allow movement outwards from Ω , but not inwards.

Whether or not such operator can be found in some analytical or compact form is beyond the scope of this thesis. It is possible, however, to find local operators for the boundaries of a rectangular simulation space. The main method is to rewrite the wave equation as a chained operation of "outgoing" (in positive direction) and "incoming" (in negative direction) wave forms. For example, for the propagation in x-direction:

$$\left(\frac{\partial^2}{\partial x^2} - \frac{1}{c^2} \frac{\partial^2}{\partial t^2}\right) \varphi = \underbrace{\left(\frac{\partial}{\partial x} - \frac{1}{c} \frac{\partial}{\partial t}\right)}_{\text{left-flowing}} \underbrace{\left(\frac{\partial}{\partial x} + \frac{1}{c} \frac{\partial}{\partial t}\right)}_{\text{right-flowing}} \varphi = 0 \quad (5.1)$$

For example, to find an operator for the left x-boundary: left-flowing waves $\varphi(x+ct)$ satisfy for the general wave equation and the left-flowing wave equation, but waves $\varphi(x-ct)$, propagating right, don't satisfy the left-flowing equation. Abstractly, if the wave equation is defined for Ω , and a limiting wave equation on $\partial\Omega$, the simulation space should remain impermeable to waves existing "outside" of the simulation. While realistically there is no "outside" in the simulation, theoretically, the boundaries also satisfying the general wave equation should provide absorbing boundary conditions. Numerical artefacts are, nevertheless, a potential wave source, and as such, limiting the direction in which waves are allowed propagate, will limit the unwanted reflections, albeit sadly not to zero.

Continuing with the previous example, to obtain a discretized left-flowing wave equation for the left x-boundary, the derivatives have to be taken at $[1/2, j]$ and at a half time-step $[q + 1/2]$

$$\frac{E_{1j}^{q+\frac{1}{2}} - E_{0j}^{q+\frac{1}{2}}}{\Delta x} = \frac{1}{c} \frac{E_{\frac{1}{2}j}^{q+1} - E_{\frac{1}{2}j}^q}{\Delta t} \quad (5.2)$$

Similar to the derivation of the update equations, averages are necessary in this case as well, since E is not defined at half-points nor half time-steps,

$$\frac{1}{2} \left(\frac{E_{1j}^{q+1} - E_{0j}^{q+1}}{\Delta x} + \frac{E_{1j}^q - E_{0j}^q}{\Delta x} \right) = \frac{1}{2c} \left(\frac{E_{1j}^{q+1} - E_{1j}^q}{\Delta t} + \frac{E_{0j}^{q+1} - E_{0j}^q}{\Delta t} \right) \quad (5.3)$$

Some algebraic manipulations yield, and taking into consideration the local wave speed at each point,

$$E_{0j}^{q+1} = E_{1j}^q + \frac{c_{0j}\Delta t - \Delta x}{c_{0j}\Delta t + \Delta x} (E_{1j}^{q+1} - E_{0j}^q) \quad (5.4)$$

defined for convenience as

$$E_{0j}^{q+1} = E_{1j}^q + C_{0j} (E_{1j}^{q+1} - E_{0j}^q) \quad (5.5)$$

The derivation of the other boundary equations is left as an exercise to the reader. Important note: to apply the boundary conditions, the electric field must already be updated in the simulation space before it can be applied on the boundary, since E^{q+1} is present on both sides of the equation. This means that tensors cannot be used in this operation, or rather, should not be used,

because it would require to store the current and previous states of the electric field, adding unnecessary overhead. A perhaps less elegant but convenient way to update the boundary is simply by array manipulations.

5.1.1 Code so far

As seen in the derivation, Mur's ABC requires the local wave speed at each point of the boundary. For this, the ABCs constants are defined for each boundary. An option to opt for the absorbing or reflecting boundary conditions is also added as a boolean. For reason that will become apparent later on, the update of each field is wrapped in a function.

```

1 import numpy as np
2 import scipy as sp
3
4 class FDTD_EM(object):
5
6     ... universal constants
7
8     def __init__(parameters go here):
9         ... parameters
10        ... fields
11        ... tensors
12        ... materials
13        ... propagation constants
14        #BOUNDARY CONDITION CONSTANTS
15        self.mur_abc = mur_abc
16        self.C0x0 = (1 / np.sqrt(self.epsilon[0, :])
17                    * self.MU_0)
18                    * self.DELTA_T - self.DELTA_X)
19                    / ((1 / np.sqrt(self.epsilon[0, :])
20                    * self.MU_0)
21                    * self.DELTA_T + self.DELTA_X))
22        self.CNx0 = (1 / np.sqrt(self.epsilon[self.SIZE_X - 1, :])
23                    * self.MU_0)
24                    * self.DELTA_T - self.DELTA_X)
25                    / ((1 / np.sqrt(self.epsilon[self.SIZE_X - 1, :])
26                    * self.MU_0)
27                    * self.DELTA_T + self.DELTA_X))
28        self.C0y0 = (1 / np.sqrt(self.epsilon[:, 0])
29                    * self.MU_0)
30                    * self.DELTA_T - self.DELTA_Y)
31                    / ((1 / np.sqrt(self.epsilon[:, 0])
32                    * self.MU_0)
33                    * self.DELTA_T + self.DELTA_Y))
34        self.CNy0 = (1 / np.sqrt(self.epsilon[:, self.SIZE_Y - 1])
35                    * self.MU_0)
36                    * self.DELTA_T - self.DELTA_Y)
37                    / ((1 / np.sqrt(self.epsilon[:, self.SIZE_Y - 1])
38                    * self.MU_0)
39                    * self.DELTA_T + self.DELTA_Y))
40
41        def __forwards_matrix(self, rows, cols): ...
42        def __backwards_matrix(self, rows, cols): ...
43        def __generate_constants(self):

```

```

44         ... update propagation constants
45         ... update ABC constants
46     def add_material(parameters go here): ...
47
48     def __updateHx(self): ... same update code
49
50     def __updateHy(self): ... same update code
51
52     def __updateEz(self): ... same update code
53
54     def update(self):
55         self.__updateHx()
56         self.__updateHy()
57         if self.mur_abc:
58             E_x_boundary = np.array([
59                 self.Ez[0,:],
60                 self.Ez[1, :],
61                 self.Ez[self.SIZE_X - 1, :],
62                 self.Ez[self.SIZE_X - 2, :]])
63             E_y_boundary = np.array([
64                 self.Ez[:,0],
65                 self.Ez[:, 1],
66                 self.Ez[:, self.SIZE_Y - 1],
67                 self.Ez[:, self.SIZE_Y - 2]])
68             self.__updateEz()
69             self.Ez[0, :] = E_x_boundary[1]
70                 + self.C0x0
71                 * (self.Ez[1, :]
72                   - E_x_boundary[0])
73             self.Ez[self.SIZE_X - 1, :] = E_x_boundary[-1]
74                 + self.CNx0
75                 * (self.Ez[self.SIZE_X - 2, :]
76                   - E_x_boundary[-2])
77             self.Ez[:, 0] = E_y_boundary[1]
78                 + self.C0y0
79                 * (self.Ez[:, 1]
80                   - E_y_boundary[0])
81             self.Ez[:, self.SIZE_Y - 1] = E_y_boundary[-1]
82                 + self.CNy0
83                 * (self.Ez[:, self.SIZE_Y - 2]
84                   - E_y_boundary[-2])
85         else:
86             self.__updateEz()

```

Chapter 6

Stability and units of measurement

This method, like any other method to solve hyperbolic partial differential equations, is subject to stability-criteria; the main thereof being the Courant Friedrichs Lewy (CFL) condition. For details regarding the general formulation, the reader may refer to other sources. A formal method to derive stability conditions for FDTD, based on the more general CFL, was presented in 1969 by Dong-Hoa Lam: *Dong-Hoa Lam (1969). "Finite Difference Methods for Electromagnetic Scattering Problems"*. Subsequently, this thesis will present an informal, yet arguably intuitive and directly applicable method based on the variables already defined, borrowing the idea from the aforementioned work.

6.1 Stability condition

The first detail to note is that all propagation constants take their biggest value when no conductivity is present ($\sigma = 0$). The "worst-case" equation for E_z becomes in this situation:

$$E_{z|ij}^{q+1} = E_{z|ij}^q + \frac{1}{\varepsilon} \left[\left(B_{ik} \frac{\Delta t}{\Delta x} \right) H_{y|kj}^q - \left(B_{ik} \frac{\Delta t}{\Delta y} \right) H_{x|jk}^q \right] \quad (6.1)$$

Substituting the H^q fields with their respective iterations yields

$$\begin{aligned}
E_{z|ij}^{q+1} &= E_{z|ij}^q \\
&+ \frac{1}{\varepsilon} \left(B_{ik} \frac{\Delta t}{\Delta x} \right) H_{y|kj}^{q-1} + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta x^2} B_{ik} F_{kl} E_{z|lj}^q \\
&- \frac{1}{\varepsilon} \left(B_{ik} \frac{\Delta t}{\Delta y} \right) H_{x|jk}^q + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta y^2} B_{ik} F_{kl} E_{z|jl}^q
\end{aligned} \tag{6.2}$$

By iterating this step back in time up to a point where no fields were yet present (assuming such initial condition) one gets a discrete integral solution to the boundary problem:

$$E_{z|ij}^{q+1} = E_{z|ij}^q + \sum_{\beta}^q \left(\frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta x^2} B_{ik} F_{kl} E_{z|lj}^{\beta} + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta y^2} B_{ik} F_{kl} E_{z|jl}^{\beta} \right) \tag{6.3}$$

Substituting the tensor operation with their results yields

$$E_{z|ij}^{q+1} = E_{z|ij}^q + \sum_{\beta}^q \left(\frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta x^2} \Omega_{z|ij}^{\beta} + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta y^2} \Lambda_{z|ij}^{\beta} \right) \tag{6.4}$$

For an arbitrary index $[ij]$ assume henceforth that $\Omega_{z|ij} \geq \Lambda_{z|ij}$. This clearly presents an upper boundary for the integrand, such that

$$\left| \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta x^2} \Omega_{z|ij}^{\beta} + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta y^2} \Lambda_{z|ij}^{\beta} \right| \leq \left| \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta x^2} + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta y^2} \right| \Omega_{z|ij}^{\beta} \tag{6.5}$$

This solution represents the accumulation of the weighted influence of the neighbouring fields on the field at $[ij]$. If each step adds a magnified influence to a point, the simulation diverges, and because the numerical method is used on every point, the instability propagates and feeds itself. To avoid this, the magnitude of the integrand must simply be bound to be less than or equal to 1. Thus,

$$\frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta x^2} + \frac{1}{\mu\varepsilon} \frac{\Delta t^2}{\Delta y^2} \leq 1 \tag{6.6}$$

$$\therefore c\Delta t \leq \frac{1}{\sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}}} \tag{6.7}$$

And for $\Delta x = \Delta y = \Delta r$

$$\sqrt{2}c\Delta t \leq \Delta r \tag{6.8}$$

6.2 Units of measurement

All values so far have been defined through their according proportions, however, yet not holding onto any particular system of units. Naturally, the default unit system would be the SI. For ease of use, this will be the chosen system. There is, however, another approach that might slightly increase the performance, but might obfuscate the units:

6.3 Normalising units

An approach to keep the numbers closer to zero is to normalise the speed of light $c = 1$, and normalize the proportion constant $\varepsilon_0 = 1$, thus making $\mu_0 = 1$ as well. An advantage of this approach is that, since μ_0 is considered constant throughout the simulation space, its multiplication can simply be omitted. The rest of the units to be derived with help of Maxwell's Equations.

6.3.1 Time and length

A standard way to derive a unit of length with the speed of light is to simply keep the unit of time as second. Then,

$$r = vt \Rightarrow [r] = cs \quad (6.9)$$

Where s is second. Choosing, for example, $\Delta t = 0.2$ and $\Delta x = 1$, yields in SI units $\Delta t = 0.2$ seconds, and $\Delta x = 1 \cdot 59958491.6$ meters.

Appendix A: Multilinear maps and tensors

Tensors and multilinear maps have a one-to-one correspondence. While a formal proof is not the scope of this section, a quick introduction of their relationship will show how to construct tensor operators for the purposes of this simulation.

6.4 Multilinear maps

6.4.1 Definition

Let V_i with $i \in \{1, 2, \dots, n\}$ and W be vector spaces over some field $\mathbb{F}$ (for our intend and purpose $\mathbb{F} = \mathbb{R}$). A multilinear map is a function of the form

$$f : V_1 \times V_2 \times \dots \times V_n \rightarrow W \quad (6.10)$$

such that f is linear on each component.

6.4.2 Unique characterization and correspondence to a tensor

The vector spaces V_i can be defined to have the dimension d_i and a set $B(V_i)$ of basis vectors $\{\mathbf{e}_{i1}, \mathbf{e}_{i2}, \dots, \mathbf{e}_{id_i}\}$. The **vector space is defined** to have the dimension d and the basis vectors $B(W) = \{\mathbf{b}_1, \dots, \mathbf{b}_d\}$. Out of each vector space V_i one basis vector can be chosen to form a new set

$$E = \left\{ (\mathbf{e}_{1j_1}, \dots, \mathbf{e}_{nj_n}) \mid (\mathbf{e}_{1j_1}, \dots, \mathbf{e}_{nj_n}) \in \prod_{k=1}^n B(V_k) \right\} \quad (6.11)$$

Define a space of multidimensional arrays as

$$\mathcal{A} = \left\{ A_{j_1, \dots, j_n}^k \mid A_{j_1, \dots, j_n}^k \in \mathbb{F}^{d \times \prod_{i=1}^n d_i} \right\} \quad (6.12)$$

There exists a unique multidimensional array, element of A , which completely characterizes the function f through the following relationship:

$$f((\mathbf{e}) \in E) = f(\mathbf{e}_{1j_1}, \dots, \mathbf{e}_{nj_n}) = \sum_{k=1}^d A_{j_1, \dots, j_n}^k \mathbf{b}_k \quad (6.13)$$

This can be proven as follows: a vector $\mathbf{v}_i \in V_i$ can be written as

$$\mathbf{v}_i = \sum_{j=1}^{d_i} v_{ij} \mathbf{e}_{ij} \quad (6.14)$$

thus, the function $f(\mathbf{v}_1, \dots, \mathbf{v}_n)$ can be expanded **on with help of** its multilinearity

$$\begin{aligned} f(\mathbf{v}_1, \dots, \mathbf{v}_n) &= f\left(\sum_{j_1=1}^{d_1} v_{1j_1} \mathbf{e}_{1j_1}, \dots, \sum_{j_n=1}^{d_n} v_{nj_n} \mathbf{e}_{nj_n}\right) \\ &= \sum_{j_1=1}^{d_1} v_{1j_1} f\left(\mathbf{e}_{1j_1}, \dots, \sum_{j_n=1}^{d_n} v_{nj_n} \mathbf{e}_{nj_n}\right) \\ &= \sum_{j_1=1}^{d_1} \dots \sum_{j_n=1}^{d_n} v_{1j_1} \dots v_{nj_n} f(\mathbf{e}_{1j_1}, \dots, \mathbf{e}_{nj_n}) \\ &= \sum_{j_1=1}^{d_1} \dots \sum_{j_n=1}^{d_n} \sum_{k=1}^d A_{j_1, \dots, j_n}^k v_{1j_1} \dots v_{nj_n} \mathbf{b}_k \end{aligned}$$



References

1. Kane Yee, "Numerical solution of initial boundary value problems involving maxwell's equations in isotropic media," in IEEE Transactions on Antennas and Propagation, vol. 14, no. 3, pp. 302-307, May 1966,.
2. Dong-Hoa Lam, "Finite Difference Methods for Electromagnetic Scattering Problems" Mississippi State University, 1969.
3. G. Mur, "Absorbing Boundary Conditions for the Finite-Difference Approximation of the Time-Domain Electromagnetic-Field Equations," in IEEE Transactions on Electromagnetic Compatibility, vol. EMC-23, no. 4, pp. 377-382, Nov. 1981.
4. John B. Schneider, "Understanding the Finite-Difference Time-Domain Method", February 2025.
5. G. Zheng, A. A. Kishk, A. W. Glisson, and A. B. Yakovlev, "Implementation of of Mur's absorbing boundaries with periodic structures to speed up the design process using Finite-Difference Time-Domain Method, University of Mississippi, 2006
6. Bulashenko, A., Piltyay, S., Dikhtyaruk, I., & Bulashenko, O. (2021). FDTD and wave matrix simulation of adjustable DBS-band waveguide polarizer. *Journal of Electromagnetic Waves and Applications*, 36(6), 875–891. <https://doi.org/10.1080/09205071.2021.1995897>
7. Jin-fa-Lee, R. Palandech and R. Mittra, "Modeling three-dimensional discontinuities in waveguides using nonorthogonal FDTD algorithm," in IEEE Transactions on Microwave Theory and Techniques, vol. 40, no. 2, pp. 346-352, Feb. 1992, doi: 10.1109/22.120108.
8. N. Khongpetch, W. Traiwattanapong, S. Chiangga, P. Limsuwan and P. Chaisakul, "FDTD Investigation of Efficient and Robust Integration Between Si3N4 and Ge-Rich GeSi for Waveguide-Integrated Electro-Absorption Optical Modulators," in IEEE Access, vol. 11, pp. 19458-19468, 2023, doi: 10.1109/ACCESS.2023.3249102.